

Neural Network Training Workflow



Theodoros Katzalis

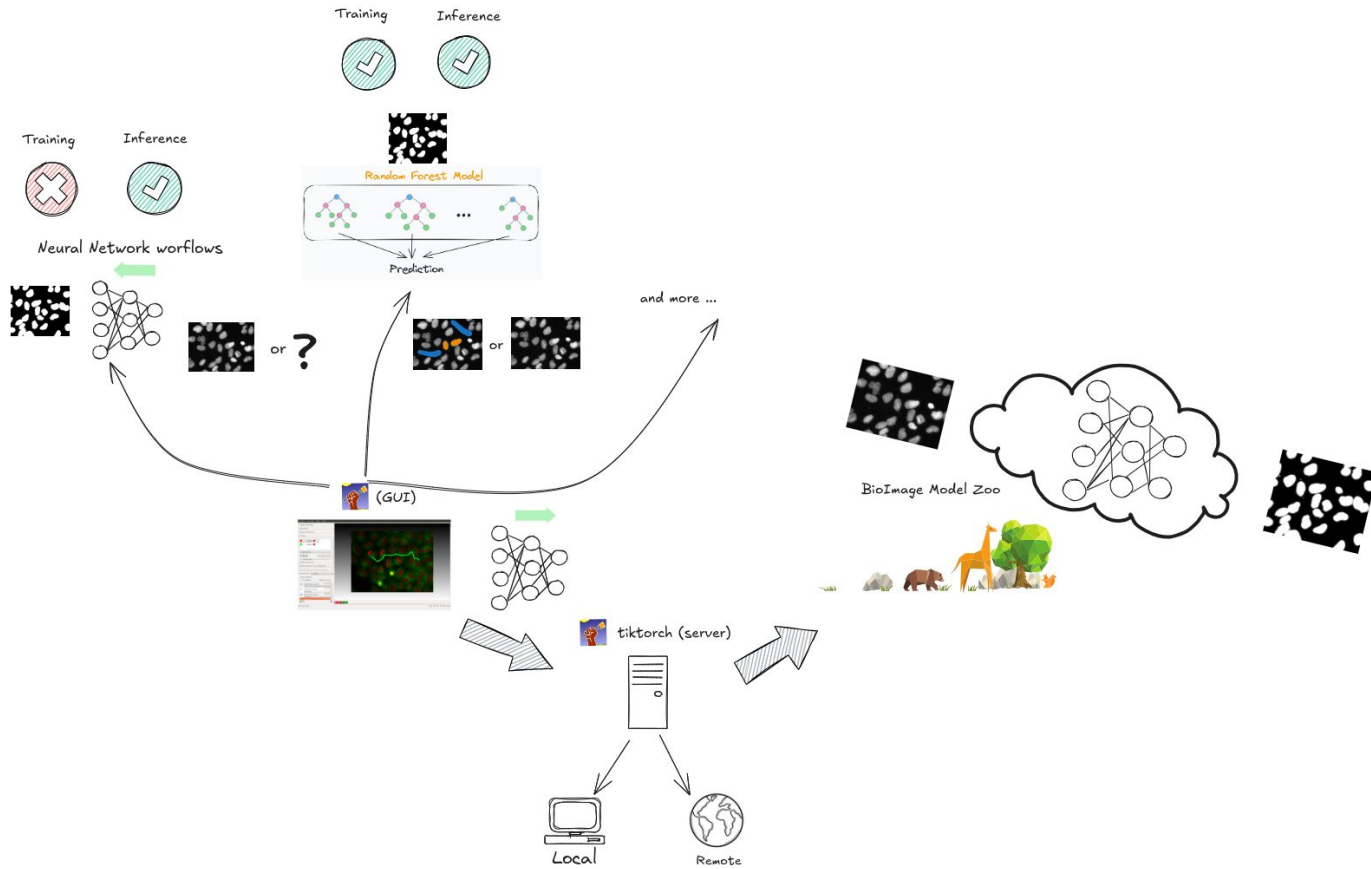
Kreshuk Group

7-12/2024

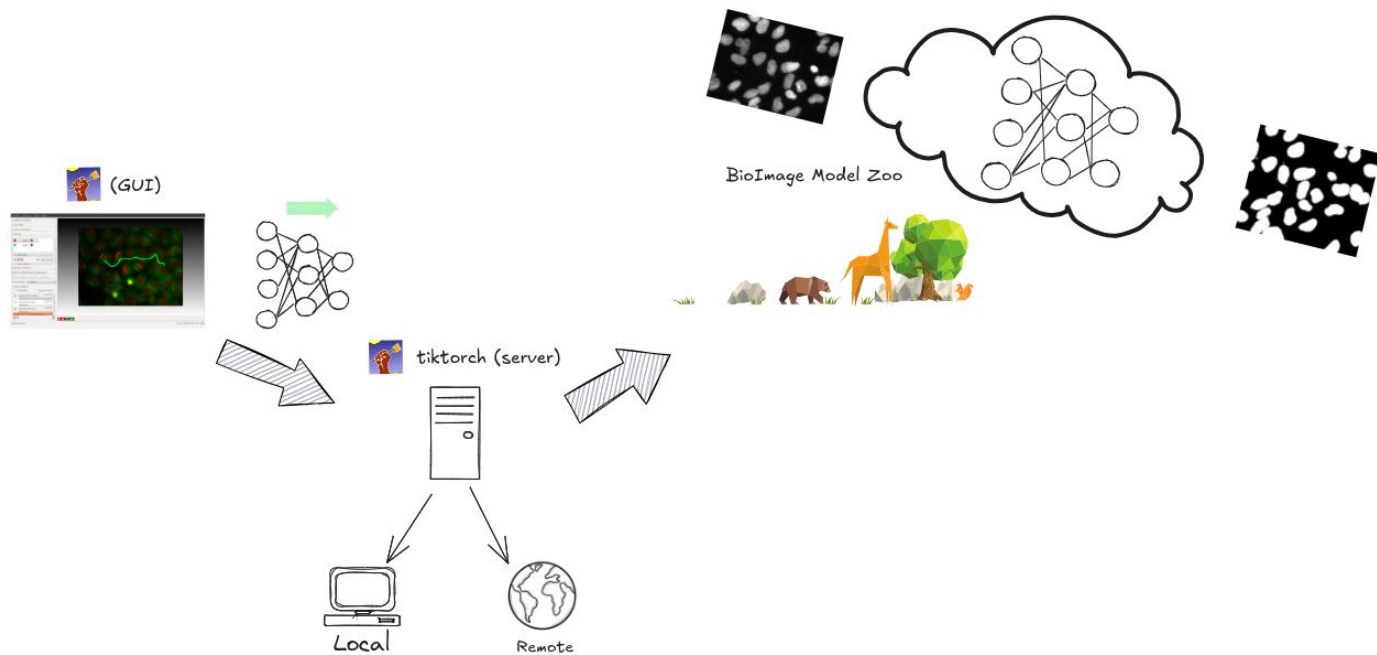
Outline

- Intro to ilastik ecosystem
- BioImage Model Zoo specification transition v4 -> v5
- Neural Network Training (tiktorch - backend)
- Surprise ?
- Next steps

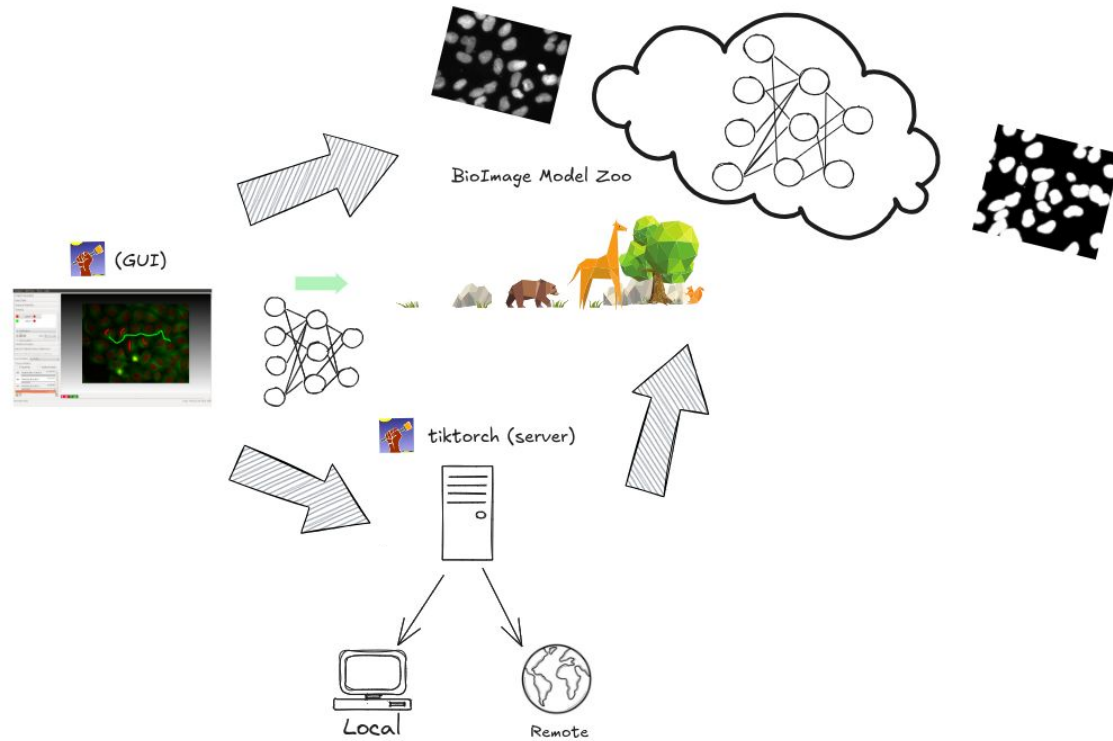
ilastik ecosystem



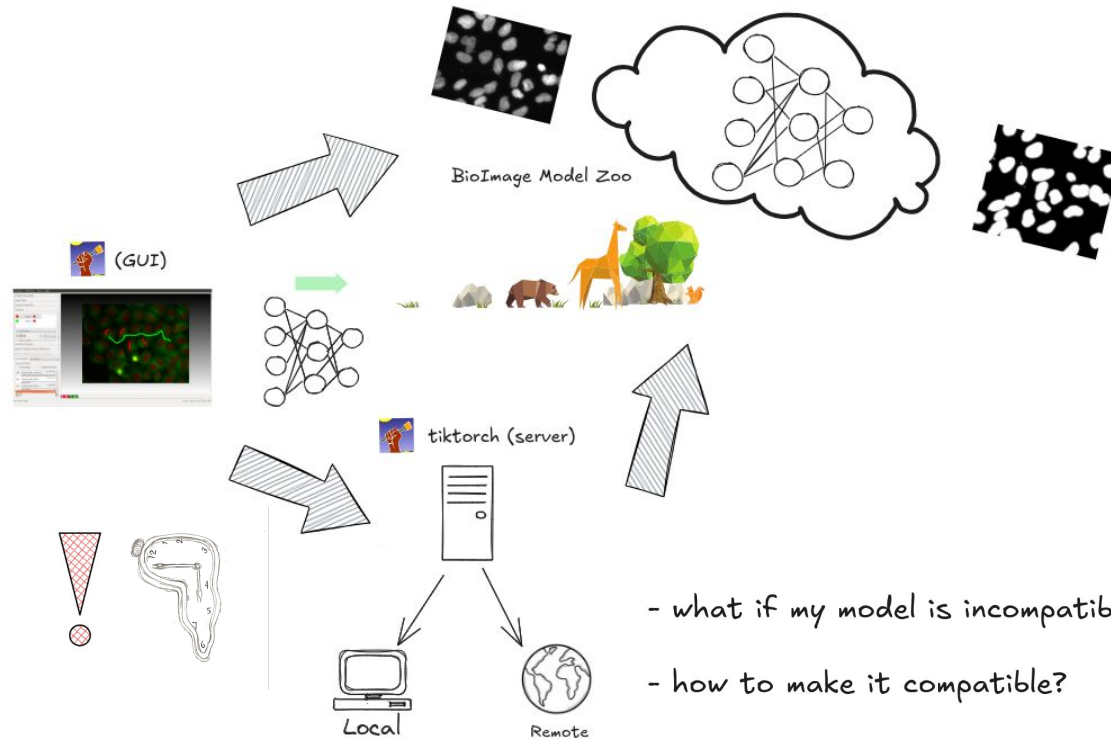
ilastik ecosystem



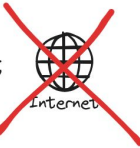
ilastik ecosystem



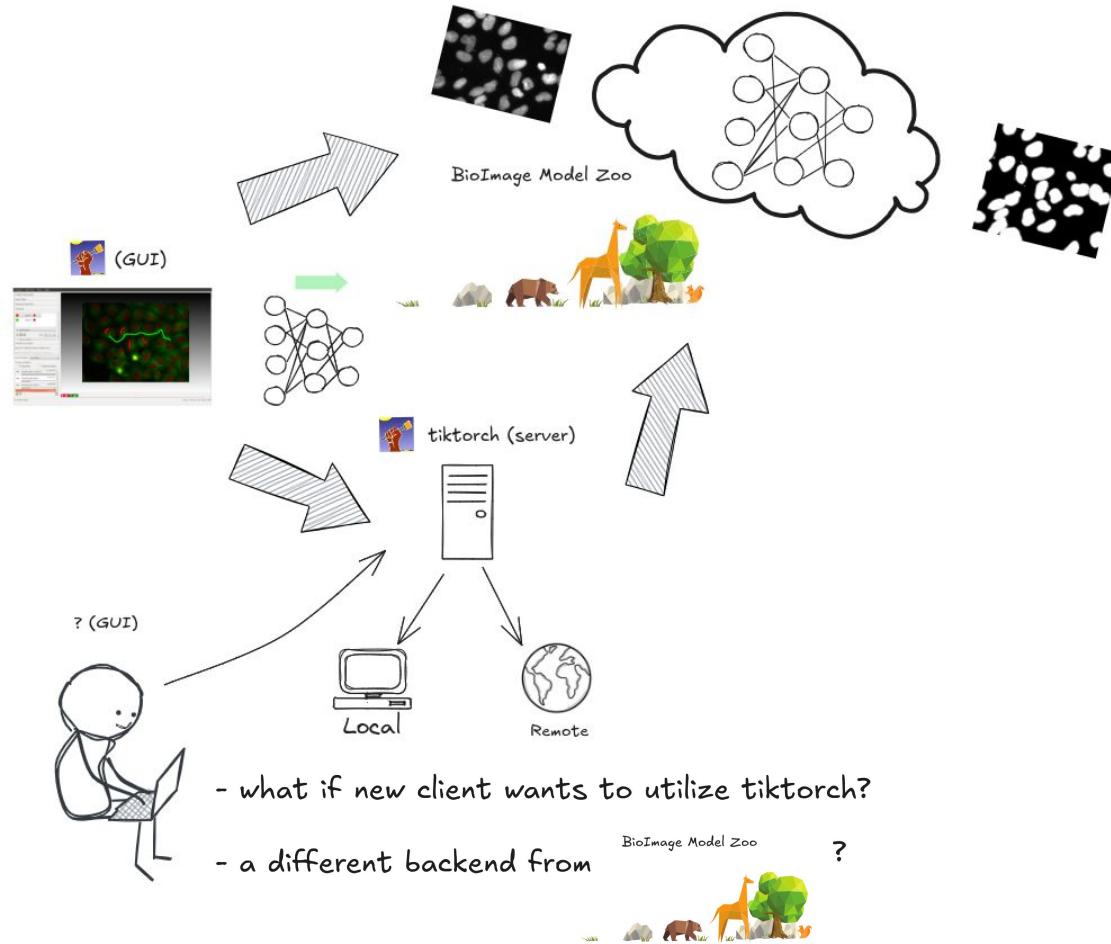
ilastik ecosystem



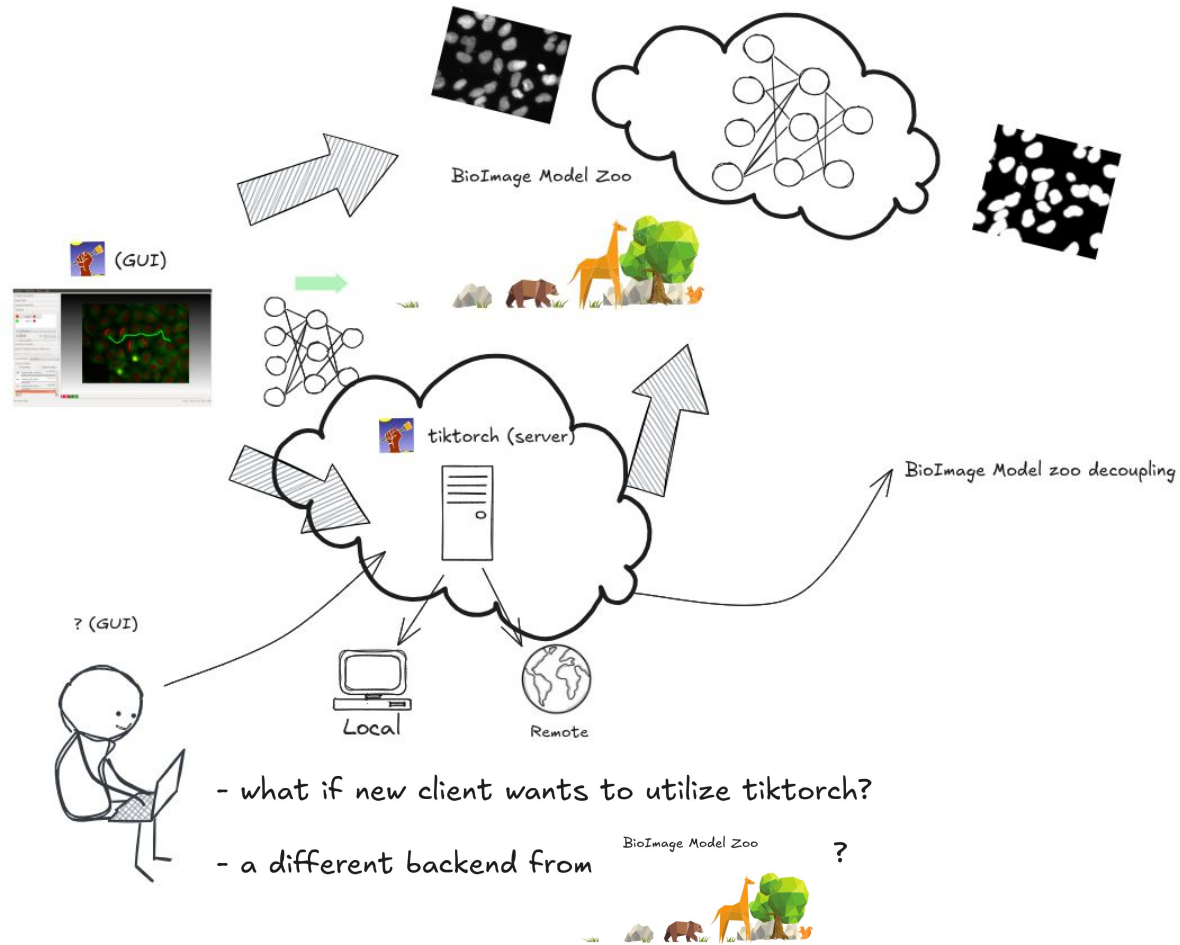
- what if my model is incompatible with my dataset?
- how to make it compatible?
- what if server can't have access to the internet



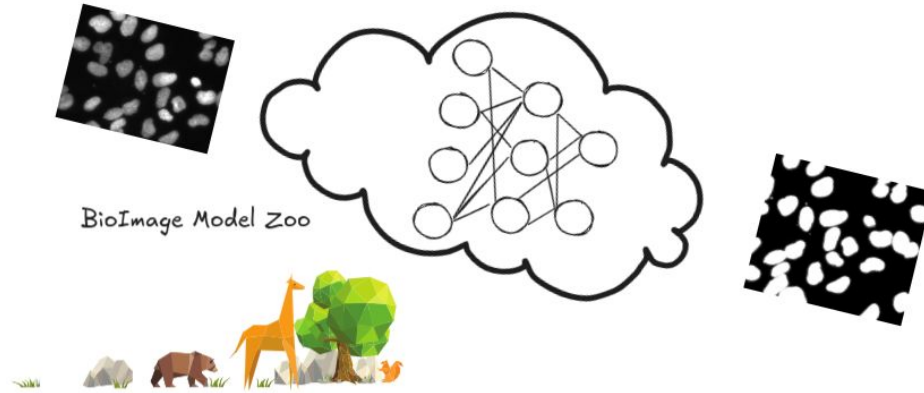
ilastik ecosystem



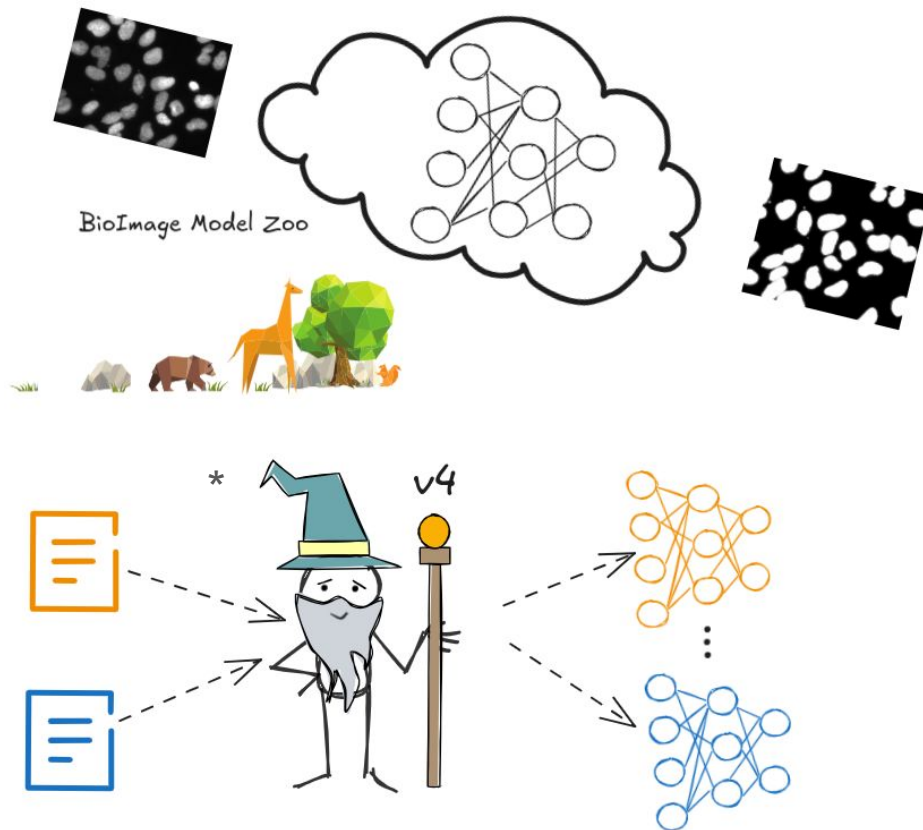
ilastik ecosystem



BioImage Model Zoo specification transition

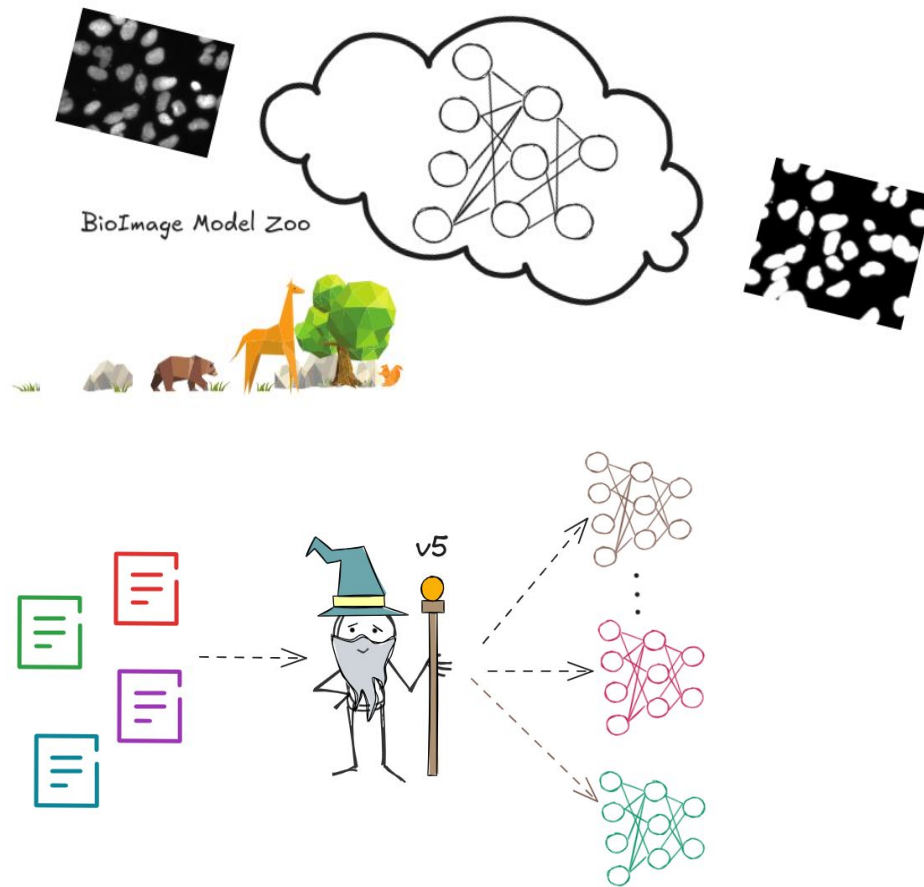


BioImage Model Zoo specification transition

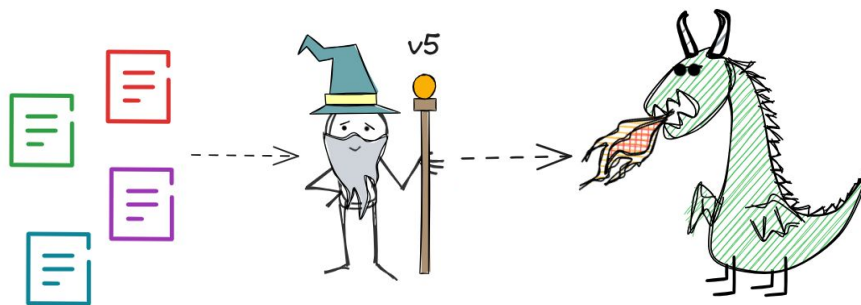
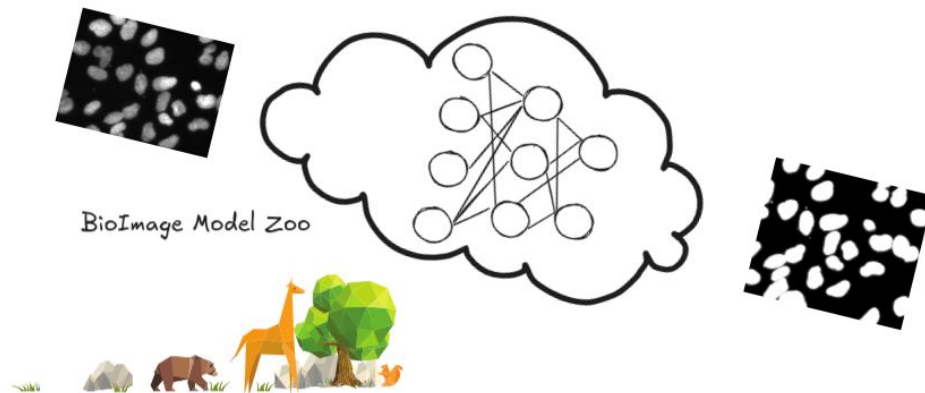


*Disclaimer: this isn't Fynn

BioImage Model Zoo specification transition



BioImage Model Zoo specification transition



BioImage Model Zoo specification transition

BioImage Model Zoo is
backwards compatible with v4 right?

But do we have
other clients?



Isn't the BioImage Model Zoo
already an abstraction?

BioImage Model Zoo specification transition

thodtack commented on Jul 9 • edited •

Mentor Author ...

The notebook contains two main sections, v4 and v5, each with code snippets and explanatory text.

v4

The input tensors could have axes with an explicit size e.g. number, that creates an explicit shape e.g. (1,2,3,4) or we can have a parameterized shape with the concept of minimum shape and step (`ParameterizedShape`).

The output tensor could have axes with explicit size as well, or an implicit one with the concept of reference tensor, scale and offset (`ImplicitOutputShape`).

v5

Instead of defining a parameterized shape for inputs or an implicit shape for outputs, this information is encapsulated per axis.

For example a tensor can have 4 axes, and per axis, we could have a parameterized (same concept with `ParameterizedShape`) size defined as `ParameterizedAxis`, a referenced size (same concept with the `ImplicitOutputShape`) defined as `ScaleReference`.

Also, more type of axis have been added that they don't fall to the category of parameterized or referenced, as shown in the sketch.

Issue

So an input tensor with one axis having size as `ParameterizedAxis`, and one more with size `ScaleReference`, cannot be resolved as either `ParameterizedShape` or `ImplicitOutputShape`, and this breaks the `MultiInfo` interface, and the `ModelSession` that provides an abstraction of bioimage that it has now.

Conclusion

I am not sure if I should proceed with a new redesign of the abstraction layer, and how this abstraction can happen. For example, it is possible the model info to have an interface only with explicit shapes, just numbers, and don't have to deal with all the implications on how axis size and shape is designed by the spec? This way it could actually create an abstraction, instead of the coupled interface that it has now.

v4 issues and design ideas #629

thodkatz opened this issue on Sep 3 · 2 comments

thodkatz commented on Sep 3 · edited ·

Hey hey!

- bloominfo.spec=0.3.3.2
- bloominfo.core=0.6.3.2

I am not sure if we still want to support v4, but I was trying to use it, for backwards compatibility for testing [networks](#), since there are some models defined as v4 sink.

I have experienced the following issues.

My project structure is:

```

├── myapp
│   ├── dummy_network.py
│   ├── init___py
│   ├── model.py
│   ├── my
│   ├── init___py
│   ├── main.py
│   └── setup.py

```

And in case you want to reproduce this setup, here is the repo <https://github.com/thodkatz/repex-playground>.

All the issues are based usually when I want to create a v4 object from scratch, not reading a yaml file.

It is worth mentioning that v5, I don't experience these issues.

Network's file path can't be absolute

```

# myapp/model.py

from myapp.dummy_model import DummyNetwork as 'DummyNetwork' in my module

dummy_model_module = os.path.dirname(os.path.dirname(__file__))

v4_vk = vk4Descriptor{
    pythonic_state_dir=os.PathJoin(os.getcwd(),
        os.path.join(weights_file_name),
        architecture+'(dummy_model_module) (DummyNetwork_name_*)',
        architecture_v4a250-'v4a250-icaf6a53f7a0b1f7fda2180135a9b0e213a406014ff4a6a4b7a'
    )
}

```

The validation will fail because absolute paths are not accepted. This can be not so convenient, if the network is defined not in a relative convenient path.

But let's try to make it relative...

Network's relative path isn't realized in model descriptor's definition

Having as reference the file structure mentioning in the beginning, expected relative path of my network's definition should be `dummy_network_name_*`

```

# myapp/model.py

v4_vk = vk4Descriptor{
    pythonic_state_dir=os.PathJoin(os.getcwd(),
        os.path.join(weights_file_name),
        architecture+'dummy_model.py (DummyNetwork_name_*)',
        architecture_v4a250-'v4a250-icaf6a53f7a0b1f7fda2180135a9b0e213a406014ff4a6a4b7a'
    )
}

```

```

# myapp/init/main.py

my_model = get_my_descriptor() # get_my_descriptor() returns the v4 object defined in 'myapp/model.py'

```

then the validation will fail with `/path/to/myapp/myapp/src/dummy_network.py doesn't point to an existing file`, meaning that the relative path was resolved based on where the module was called, and not defined.

But even if this is desired behavior, we can fix this by using `../dummy_model.py`, but then the next issue will come.

Serialized object keeps relative path

```
# myspec/model.py
v0_v4_WeightsDescr(
    pytorch_state_dict=v0_4_PytorchStateDictWeightsDescr(
        source_path(weights_file_name),
        architecture="dummy_model.py [DummyNetwork, _name_]",
        architecture_sha256="cble3081cc44d03f3acd91cf735d12863150d4bd213d460614fe084ebbf9a"
    )
)

# myspec/src/main.py
from myspec.model import get_binimage_model_v4
from binimageio.spec import save_binimageio_package, load_description

model = get_binimage_model_v4()
zip_file = save_binimageio_package(model)
ret_model = load_description(zip_file)

, the load_description will fail with /tmp/tpubbykms binimageio.zip unzip: ./dummy_network.py, meaning that the
relative path of ./dummy_network was retained when we have attempted to save the package with
save_binimageio_package. I think it should be updated based on the structure that the save_binimageio_package is
crafting.
```

What I currently use and works nicely

providing modules in the architecture, works nicely.) It is imported as follows.

```
weights_v0_4_WeightsDescr(
    pytorch_state_dict=v0_4_PytorchStateDictWeightsDescr(
        source_path(weights_file_name),
        architecture="myspec.dummy_network.DummyNetwork",
    )
)
```

Idea

Also, regarding the design of the v5 and v4 models, I think that they are very much coupled with files, and an expected yaml file as a reference point. Maybe an object ModelDescr should be assumed as the ground truth, and instead of using an object to describe a file, the object should just be expressed as a file.

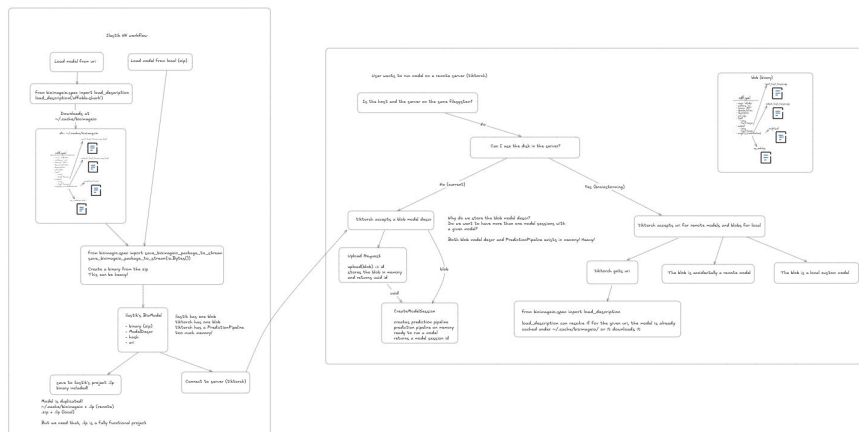
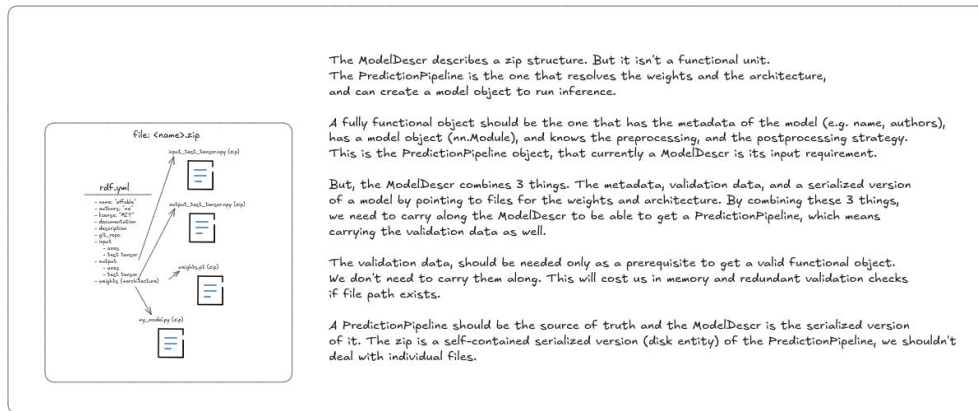
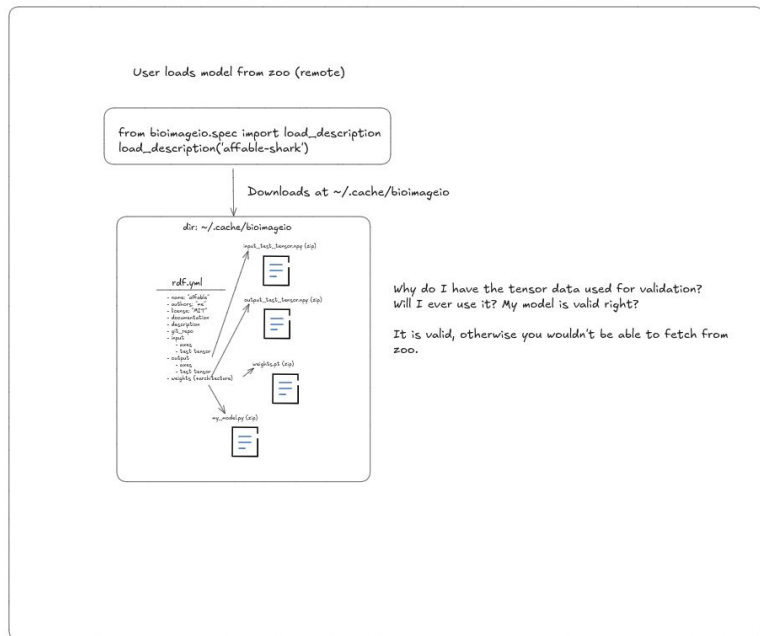
Also instead of relying on a path to get to the data, the object should be content dependent. For example, if we have a dependency, a numpy array, then we need a numpy array and not a file that points to a numpy array. That way the dependency is decoupled, and there could be many ways to get a numpy array.

I am trying to say that maybe creating a ModelDescr object should be possible without any file? We could have a nice decoupling, and testing won't rely on disk operations as well.

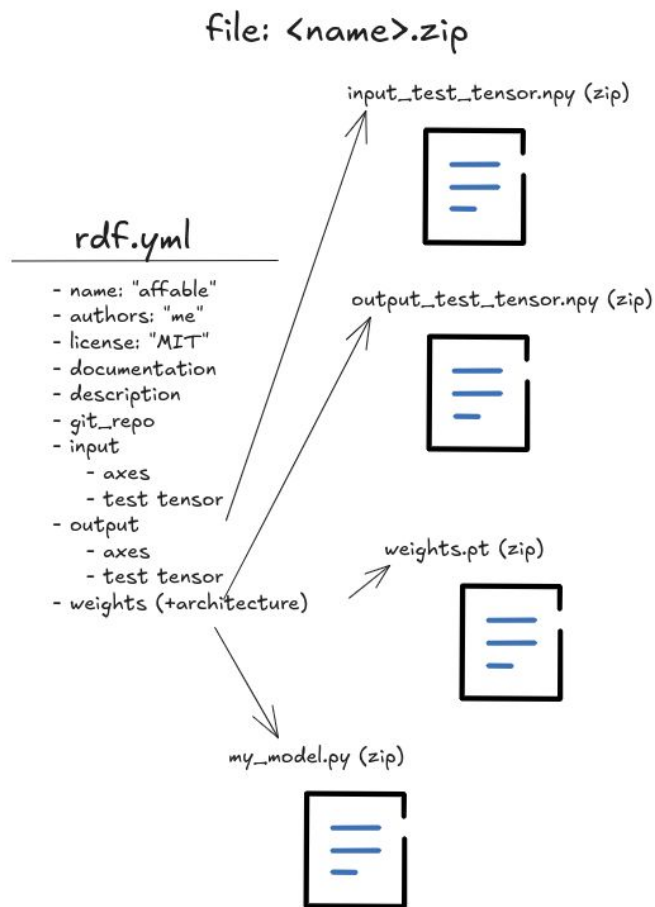
From a design perspective, a builder pattern could be also very useful, since these objects have many arguments, and some could be optional, making the process of creating objects like that quite sophisticated. A builder pattern could be very handy. I am not sure if pydantic supports this.

edit: the builder pattern isn't really valid, essentially the features of optional, keyword, required arguments with potential classmethods provide the functionality of a builder pattern. Python supports all of the above. Nice read: <https://python-patterns.guide/nlp-of-four-builder/>

BiolImage Model Zoo specification transition



BioImage Model Zoo specification transition



BioImage Model Zoo specification transition

Okey, but what has been done eventually?

BioImage Model Zoo specification transition

checklist



Remove redundant abstractions

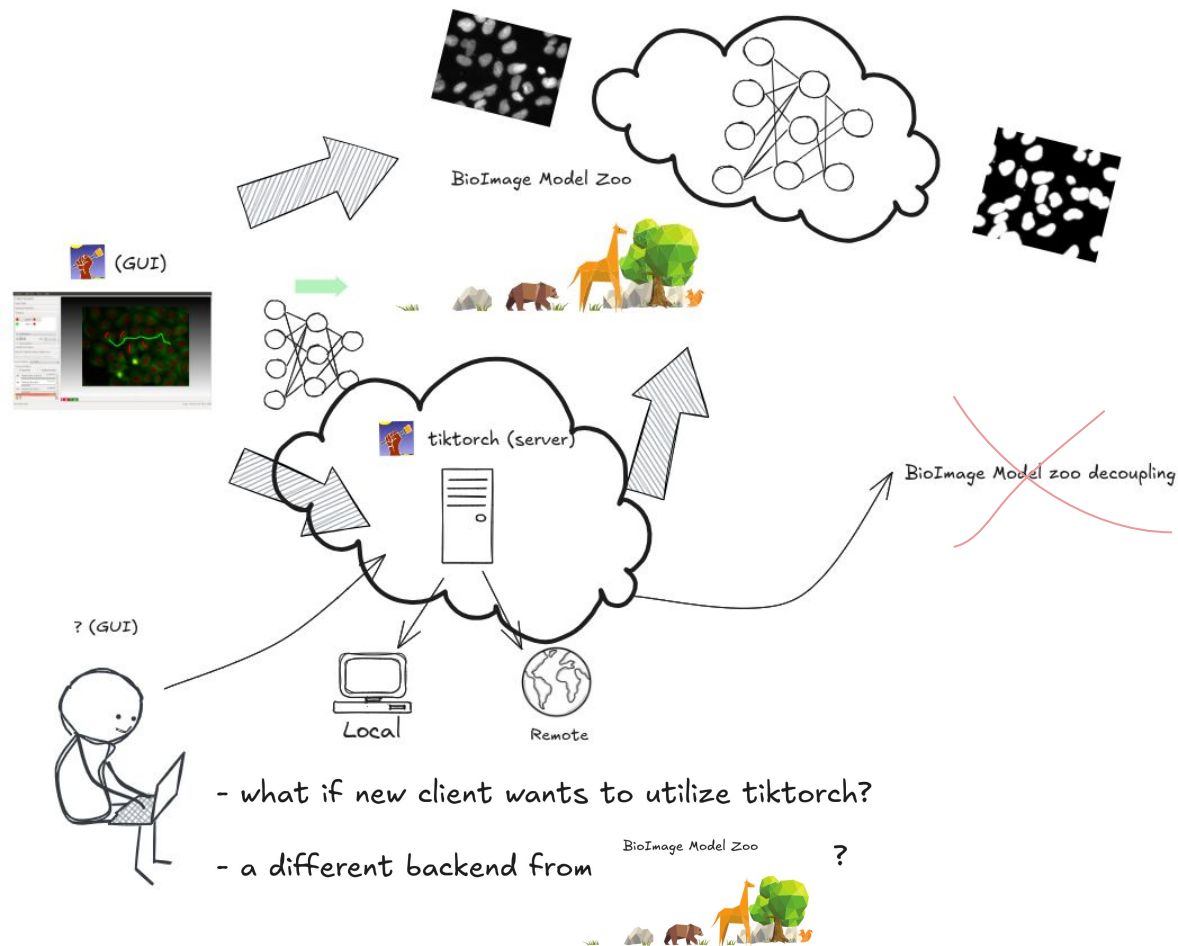


v4 -> v5 (axes, shapes, and all that jazz)



Input validation (can I forward this tensor to my model?)

BioImage Model Zoo specification transition



BioImage Model Zoo specification transition

spec v4

InputTensor

- name: e.g. "input"
- shape: e.g. (1,2,3,4) or
ParameterizedShape(min=[1,2,3,4],
step=[0.1,0.2,0.3,0.4])
- axes: e.g. ('b','c','x','y')

OutputTensor

- name: e.g. "ouput"
- shape: e.g. (1,2,3,4) or
ImplicitOutputShape(reference_tensor="input",
scale=[1,2,3,4],
offset=[0,0,0,0])
- halo: e.g. None or [10,10,10,10]

BioImage Model Zoo specification transition

spec v5

InputTensor

- tensor_id
- axes: e.g. list[axis] where axis: e.g.
 - BatchAxis()
 - ChannelAxis()
 - IndexInputAxis()
 - TimeInputAxis(),
 - SpaceInputAxis()

OutputTensor

- tensor_id
- axes: e.g. list[axis] where axis e.g.
 - BatchAxis()
 - ChannelAxis()
 - IndexOutputAxis()
 - TimeOutputAxis() with or without halo
 - SpaceOutputAxis() with or without halo

BatchAxis

- size: Optional[Literal[1]]

ChannelAxis

- channel_names: list
- size: len(channel_names)

IndexInputAxis

- size: fixed integer or
ParameterizedSize(min=1, step=0.1) or
SizeReference(tensor_id, axis_id, offset, scale)

TimeInputAxis

- size same as IndexInputAxis

SpaceInputAxis

- size same as IndexInputAxis

IndexOutputAxis

- size: fixed integer or
SizeReference(tensor_id, axis_id, offset, scale)
DataDependentSize(min, max)

TimeOutputAxis with Halo

- size: SizeReference(tensor_id, axis_id, offset, scale)

TimeOutputAxis

- size: fixed integer or
SizeReference(tensor_id, axis_id, offset, scale)

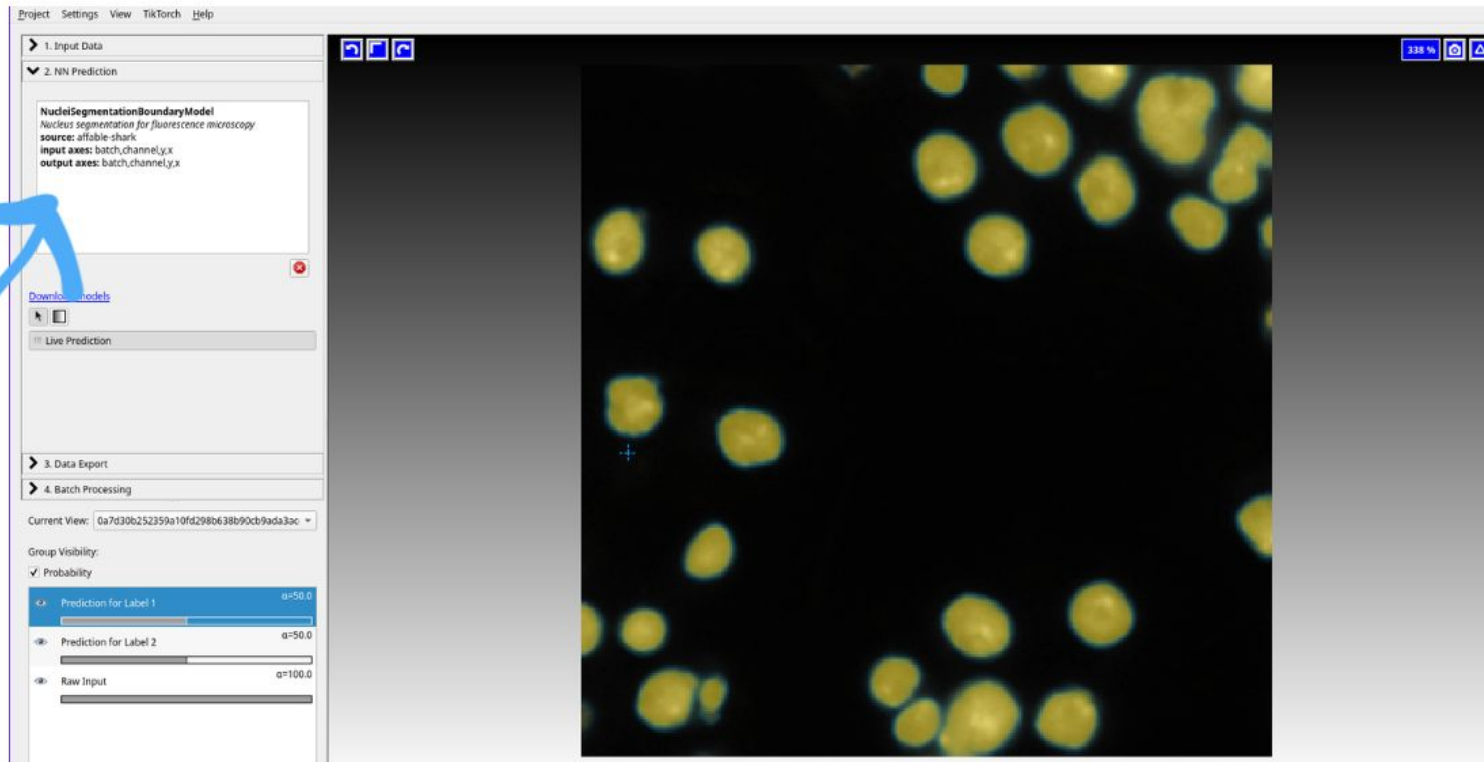
SpaceOutputAxis with Halo

- size: same as TimeOutputAxis with Halo

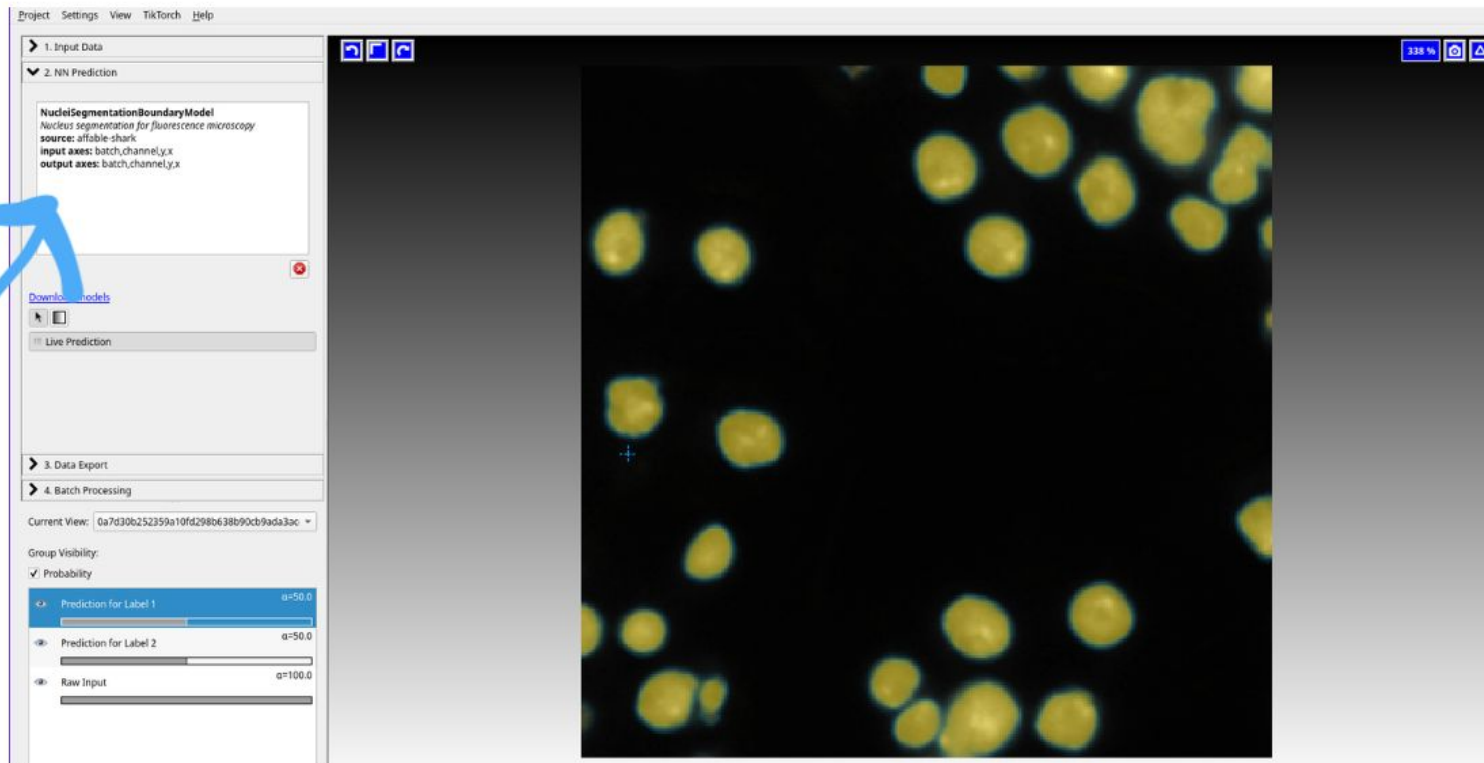
SpaceOutputAxis

- same as TimeOutputAxis

BioImage Model Zoo specification transition



Demo!



Shallow2Deep

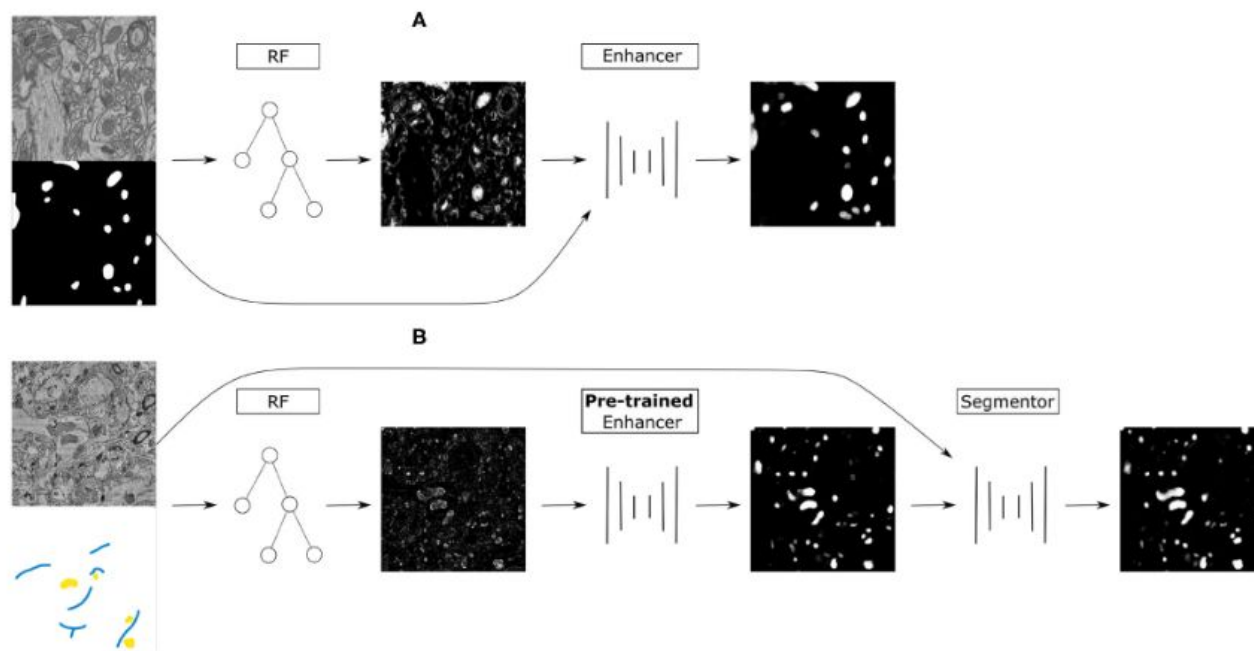


FIGURE 1 | (A) Training on the source dataset: many Random Forests are trained by subsampling patches of raw data and dense groundtruth segmentation. Random Forest predictions are used as inputs and groundtruth segmentation as labels to train the Prediction Enhancer CNN to improve RF segmentations. **(B)** Domain adaptation to the target dataset: a RF is trained interactively with brushstroke labels. The pre-trained PE is applied to improve the RF predictions. Optionally, PE predictions are used as pseudo-labels to train a segmentation network for even better results with no additional annotations, but using a larger computational budget.

Deep learning frameworks for training



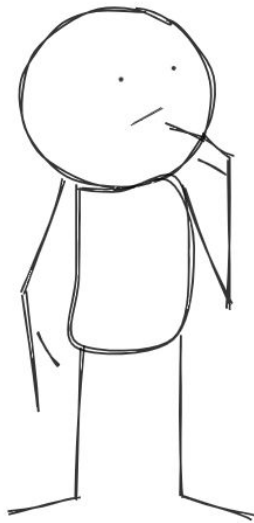
Medical Image Segmentation
with Convolutional Neural Networks

Deep learning frameworks for training

What is the abstraction layer?

Do you see a pattern?

Can I choose one framework?



Deep learning frameworks for training

pytorch-3dunet



pytorch-3dunet interface

pytorch-3dunet on tiktorch

- Init
- Start
- Pause
- Resume
- Forward
- Save
- Export to bioimageio model zoo (.zip)
- Get training state (idle, paused, running, failed)
- Best model notification

Concerns

- Halo
- Input shapes
- Preprocessing, postprocessing compatibility of pytorch-3dunet and bioimageio
- Torchscript compatibility
- Configuration objects, yaml decoupling
- Trainer of pytorch-3dunet, good reference, but do we stick with pytorch 3d unet?
- Pydantic, convenient for serialization, deserialization
- Export kwargs issue